Київський національний університет імені Тараса Шевченка
Факультет інформаційних технологій
Кафедра програмних систем і технологій

ЗВІТ

до лабораторної роботи
з дисципліни «Web-технології»

Розроблення платформи спільної роботи StudySpace

Виконав:
студент групи ТТП-32
Кашинський Владислав

Київ – 2026

Анотація

У лабораторній роботі спроєктовано та реалізовано StudySpace, навчальну web-платформу для організації робочих просторів, документів, файлів і спільної роботи. Пов’язані вимоги об’єднано в один Git-монорепозиторій, а технологічно незалежні частини оформлено як окремі застосунки. Рішення містить ASP.NET Core MVC/API, React SPA, Azure Functions, RabbitMQ Worker, MCP-сервер, SQLite, SignalR, SSE, WebSocket, Lucene.NET, Leaflet/OpenStreetMap, Chart.js та автоматизовані тести.

Обрано 36 пунктів завдання сумарною вартістю 104 бали. Під час перевірки підтверджено форматування і збірку .NET-рішення, 20 модульних та інтеграційних тестів, 5 browser E2E-сценаріїв, production-збірку React, REST CRUD, пагінацію, кеш, пошук, SSE і MCP. Зовнішні cloud-інтеграції залишено вимкненими до додавання особистих облікових даних студента.

Ключові слова: ASP.NET Core, MVC, REST API, React, SignalR, WebSocket, SSE, SQLite, Entity Framework Core, RabbitMQ, Azure Functions, MCP, Playwright, Docker.

Зміст

Анотація **2**

1 Постановка задачі **6**

1.1 Мета роботи . . . 6

1.2 Предметна область . . . 6

1.3 Вибір завдань і розрахунок . . . 6

2 Поділ рішення на проєкти **7**

2.1 Чому монорепозиторій . . . 7

2.2 Запускні застосунки . . . 7

2.3 Усі технічні проєкти . . . 7

2.4 Структура каталогів . . . 7

3 Архітектура і модель даних **8**

3.1 Архітектурна схема . . . 8

3.2 Сутності . . . 9

3.3 Ключові маршрути . . . 9

4 Послідовність роботи системи **9**

4.1 Крок 1. Запуск і dashboard . . . 9

4.2 Крок 2. Робочі простори . . . 10

4.3 Крок 3. Конкретний workspace . . . 11

4.4 Крок 4. Створення документа . . . 12

4.5 Крок 5. Деталі документа . . . 13

4.6 Крок 6. Блоковий редактор . . . 14

4.7 Крок 7. Файли . . . 15

4.8 Крок 8. Аналітика . . . 16

4.9 Крок 9. REST API . . . 17

4.10 Крок 10. Мережеві механізми . . . 18

4.11 Крок 11. Спільна дошка . . . 19

4.12 Крок 12. Карта . . . 20

4.13 Крок 13. SSE-помічник . . . 20

5 Детальний опис 36 завдань **21**

5.1 Common . . . 21

5.1.1 C1: MVC і модель даних (3 б.) . . . 21

5.1.2 C2: Azure deployment (1 б.) . . . 21

5.1.3 C3: Continuous integration (2 б.) . . . 21

5.1.4 C4: Контейнеризація (2 б.) . . . 22
5.1.5 C5: REST API (3 б.) . . . 22
5.1.6 C6: Адаптивний UI (3 б.) . . . 22
5.1.7 C7: SEO (2 б.) . . . 22
5.1.8 C8: Карта (3 б.) . . . 22
5.1.9 C9: Application Insights (2 б.) . . . 22
5.1.10 C10: Polling, long polling, WebSocket (4 б.) . . . 22
5.1.11 C11: SignalR (4 б.) . . . 23
5.1.12 C12: Блоковий редактор (3 б.) . . . 23
5.1.13 C13: Realtime content (2 б.) . . . 23
5.1.14 C14: Presence і focus (2 б.) . . . 23
5.1.15 C15: File API (3 б.) . . . 23
5.1.16 C16: OAuth 2.0 (3 б.) . . . 23
5.1.17 C18: Realtime shared board (4 б.) . . . 23
5.1.18 C19: Cloud/local storage (3 б.) . . . 24
5.1.19 C21: SSE-помічник (5 б.) . . . 24
5.1.20 C22: MCP server (5 б.) . . . 24
5.2 BackEnd . . . 24
5.2.1 B1: Пагінація (2 б.) . . . 24
5.2.2 B2: Webhook/Telegram (2 б.) . . . 24
5.2.3 B4: Full-text search (2 б.) . . . 24
5.2.4 B5: Ролі (3 б.) . . . 25
5.2.5 B6: Multi-tenancy і plans (3 б.) . . . 25
5.2.6 B7: Azure Functions (3 б.) . . . 25
5.2.7 B8: RabbitMQ worker (4 б.) . . . 25
5.2.8 B9: API cache (3 б.) . . . 25
5.3 FrontEnd . . . 25
5.3.1 F1: Chart.js (3 б.) . . . 25
5.3.2 F2: jQuery autocomplete (2 б.) . . . 25
5.3.3 F3: Drag-and-drop (2 б.) . . . 26
5.3.4 F5: React SPA (3 б.) . . . 26
5.3.5 F6: Pointer Events і Canvas (5 б.) . . . 26
5.4 QA . . . 26
5.4.1 QA1: API integration tests (2 б.) . . . 26
5.4.2 QA2: Browser E2E (3 б.) . . . 26
5.4.3 QA3: Authorization tests (3 б.) . . . 26

6 REST API і ручна перевірка 26

7 Безпека і конфігурація **27**

8 Тестування та результати перевірки **28**

8.1 MCP smoke test 29

9 Запуск кожного застосунку **29**

9.1 Web 29

9.2 React SPA 29

9.3 Container stack 29

9.4 Azure Functions 29

9.5 MCP 29

10 Автоматизація перевірок **30**

11 Висновки **30**

1. Постановка задачі

1.1. Мета роботи

Мета роботи полягає у проходженні повного циклу створення web-системи: від моделі даних і CRUD до REST API, браузерного інтерфейсу, авторизації, realtime-обміну, інтеграцій, тестування та підготовки розгортання. Практичний результат має працювати локально й бути готовим до подальшої публікації.

1.2. Предметна область

StudySpace зберігає навчальні матеріали у робочих просторах. Простір має назву, опис, тариф, координати та учасників. У ньому створюються документи зі статусом, summary, обкладинкою, впорядкованими блоками та файлами. Учасники мають ролі Owner, Editor або Viewer. Premium відкриває дошку, помічника та розширені можливості.

1.3. Вибір завдань і розрахунок

Сума балів за реалізовані завдання становить 104. За формулою з умови номінальний результат:

$$60 \cdot \frac{104}{100} = 62{,}4.$$

Етапи	Практичний результат	Бали
C1, C5, QA1, B1	MVC CRUD, файли, REST, Swagger/Postman, пагінація	10
C2, C3, C4	Azure-ready deployment, GitHub Actions, Docker/Compose	5
C6, C7, F1, F2, QA2	Landing, SEO, Chart.js, AJAX, E2E	13
C8, B2, C9, B4	Карта, webhook/Telegram, telemetry, Lucene	9
C10, C11	Polling, long polling, WebSocket, SignalR	8
C12, F3, C13, C14	Block editor, drag-and-drop, realtime, presence	9
C15	File API, DataTransfer, FileReader	3
C16, B5, B6, QA3	OAuth, ролі, multi-tenancy, plans, auth tests	12
F5	Окремий React + TypeScript SPA	3
B7	Окремий Azure Functions API	3
C18, F6	Shared board, Pointer Events, Canvas 2D	9
B8, B9, C19	RabbitMQ, memory cache, Azure Blob/local storage	10
C21, C22	SSE-помічник і MCP tools	10
	Разом	**104**

2. Поділ рішення на проєкти

2.1. Чому монорепозиторій

Усі частини працюють з однією предметною областю, тому їх збережено в одному репозиторії. Це дає спільну історію, одну команду збірки й відсутність дублювання моделей. Водночас кожен незалежний процес є окремим проєктом. Результат: **один репозиторій**, **5 запускних застосунків** і **10 технічних проєктів**.

2.2. Запускні застосунки

Застосунок	Розташування	Роль
StudySpace Web	`src/StudySpace.Web`	MVC, REST, SignalR, SSE, WebSocket, UI
StudySpace SPA	`src/studyspace-spa`	React/TypeScript REST-клієнт
StudySpace Functions	`src/StudySpace.Functions`	Serverless health і digest
StudySpace Worker	`src/StudySpace.Worker`	RabbitMQ consumer
StudySpace MCP	`src/StudySpace.Mcp`	Stdio JSON-RPC/MCP tools

Таблиця 2: П’ять окремо запущуваних частин

2.3. Усі технічні проєкти

Проєкт	Призначення	Тип
StudySpace.Core	Сутності й enum предметної області	.NET library
StudySpace.Data	DbContext, EF Core, SQLite, migration, seed	.NET library
StudySpace.Web	MVC, API, realtime, UI, integrations	.NET web
StudySpace.Functions	Ізольовані Azure Functions	.NET functions
StudySpace.Worker	RabbitMQ background service	.NET worker
StudySpace.Mcp	MCP stdio server	.NET console
StudySpace.UnitTests	Доменні unit-тести	.NET tests
StudySpace.IntegrationTests	API й authorization tests	.NET tests
studyspace-spa	React, TypeScript, Vite	Node.js
e2e	Playwright Chromium scenarios	Node.js

2.4. Структура каталогів

Лістинг 1: Скорочена структура

```
StudySpace/
|-- src/
|   |-- StudySpace.Core/        |-- StudySpace.Data/
|   |-- StudySpace.Web/         |-- StudySpace.Functions/
|   |-- StudySpace.Worker/      |-- StudySpace.Mcp/
|   `-- studyspace-spa/
|-- tests/
|   |-- StudySpace.UnitTests/   |-- StudySpace.IntegrationTests/
|   `-- e2e/
|-- docs/stages/                |-- report/WebTechnologiesLab/
|-- .github/workflows/ci.yml    |-- compose.yaml
`-- StudySpace.slnx
```

3. Архітектура і модель даних

3.1. Архітектурна схема

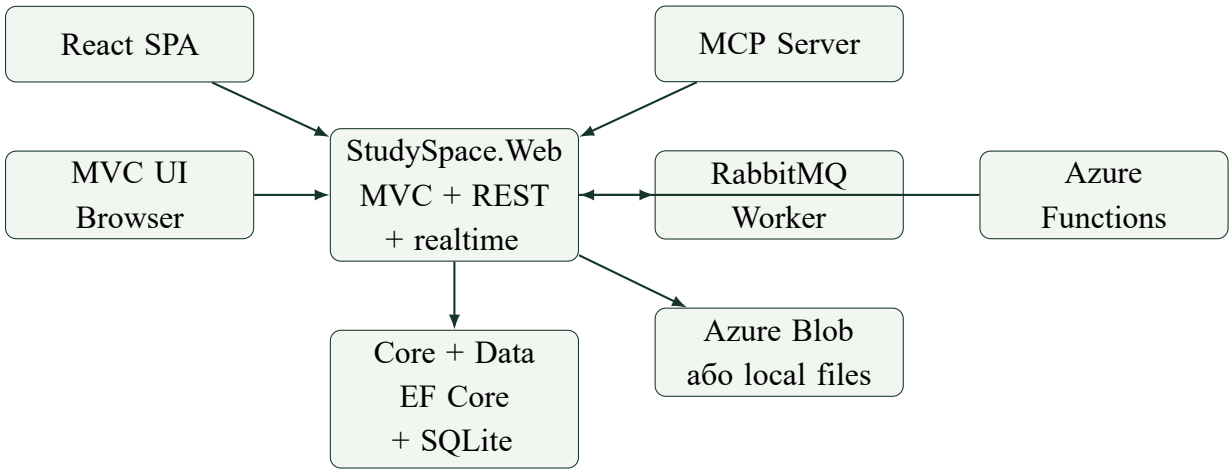


Рис. 1: Взаємодія компонентів StudySpace

Основна точка входу налаштовує dependency injection, EF Core, cookies, опційний Google OAuth, Swagger, SignalR, Application Insights і middleware. Контролери перед читанням або зміною даних виконують tenant-aware перевірку.

3.2. Сутності

Сутність	Основні поля та зв’язки
Workspace	Id, Name, Description, Plan, coordinates; Documents і Members
Document	WorkspaceId, Title, Summary, CoverImageUrl, Status; Blocks, Attachments
DocumentBlock	DocumentId, Type, Content, Position
Attachment	DocumentId, FileName, ContentType, Size, Url
WorkspaceMember	WorkspaceId, UserId, DisplayName, Role

Таблиця 4: Модель предметної області

SQLite обрано для запуску без окремого сервера. EF Core створює схему через міграції. Seed ідемпотентний: демонстраційні дані додаються лише у порожню БД.

3.3. Ключові маршрути

Маршрут	Призначення
`/`	dashboard
`/Workspaces`	список і CRUD просторів
`/Documents/Details/{id}`	документ, блоки, файли
`/Documents/Editor/{id}`	realtime editor
`/Files`	бібліотека вкладень
`/Insights`	статистика й Chart.js
`/Connectivity`	три способи оновлення
`/Board`	Canvas-дошка
`/Map`	Leaflet/OpenStreetMap
`/swagger`	REST документація
`/api/assistant/stream`	SSE-помічник

Таблиця 5: Ключові маршрути

4. Послідовність роботи системи

Знімки зроблено під час фактичного локального запуску `http://localhost:5036`; це не макети. Інтерфейс застосунку локалізовано українською мовою.

4.1. Крок 1. Запуск і dashboard

Під час запуску застосовуються міграції та seed. Dashboard показує кількість просторів, документів, опублікованих матеріалів і файлів.

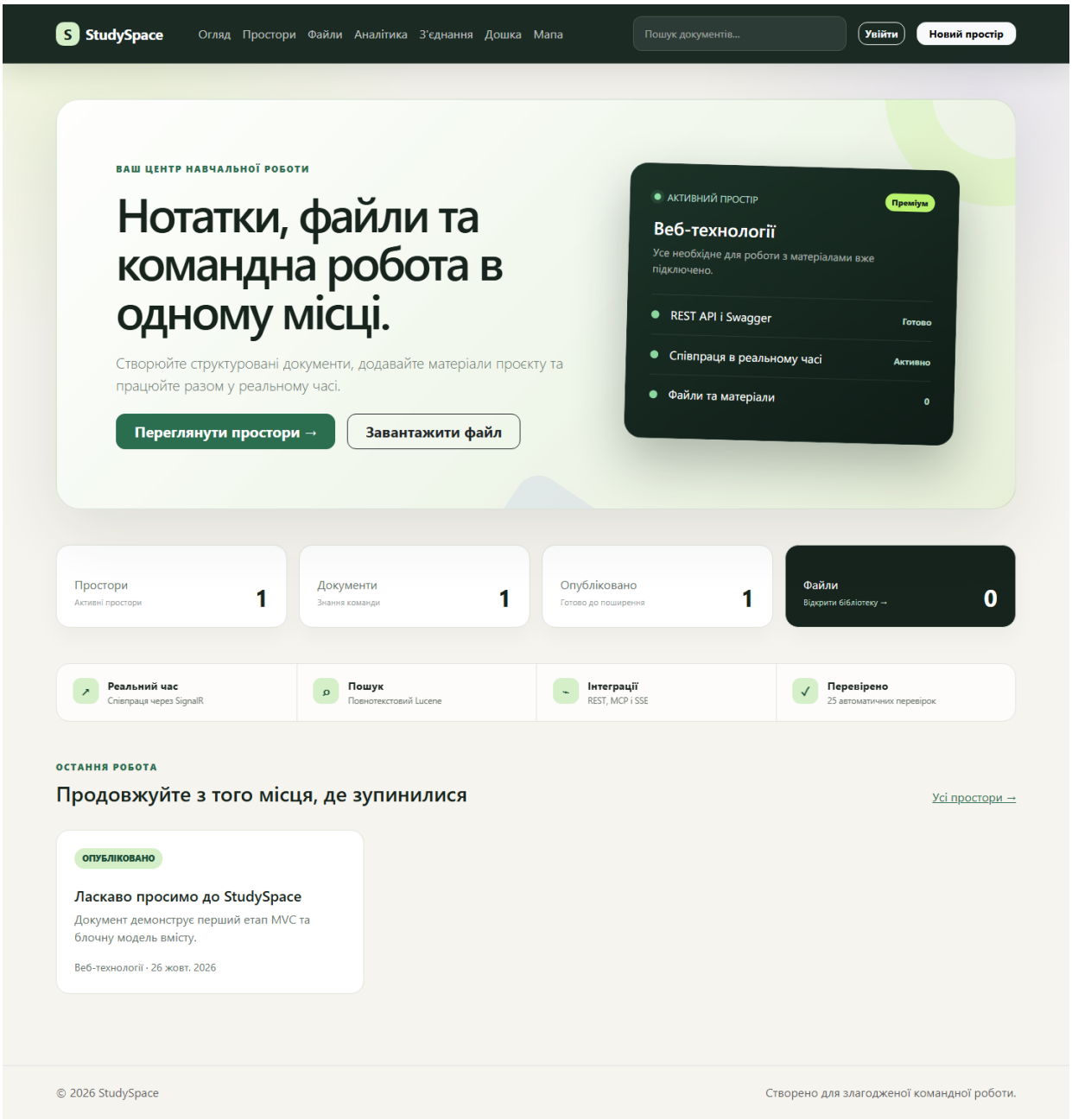


Рис. 2: Головна сторінка після локального запуску

4.2. Крок 2. Робочі простори

Список фільтрується за поточним користувачем. Картка містить тариф, опис і кількість документів; кнопка створення простору відкриває валідовану MVC-форму.

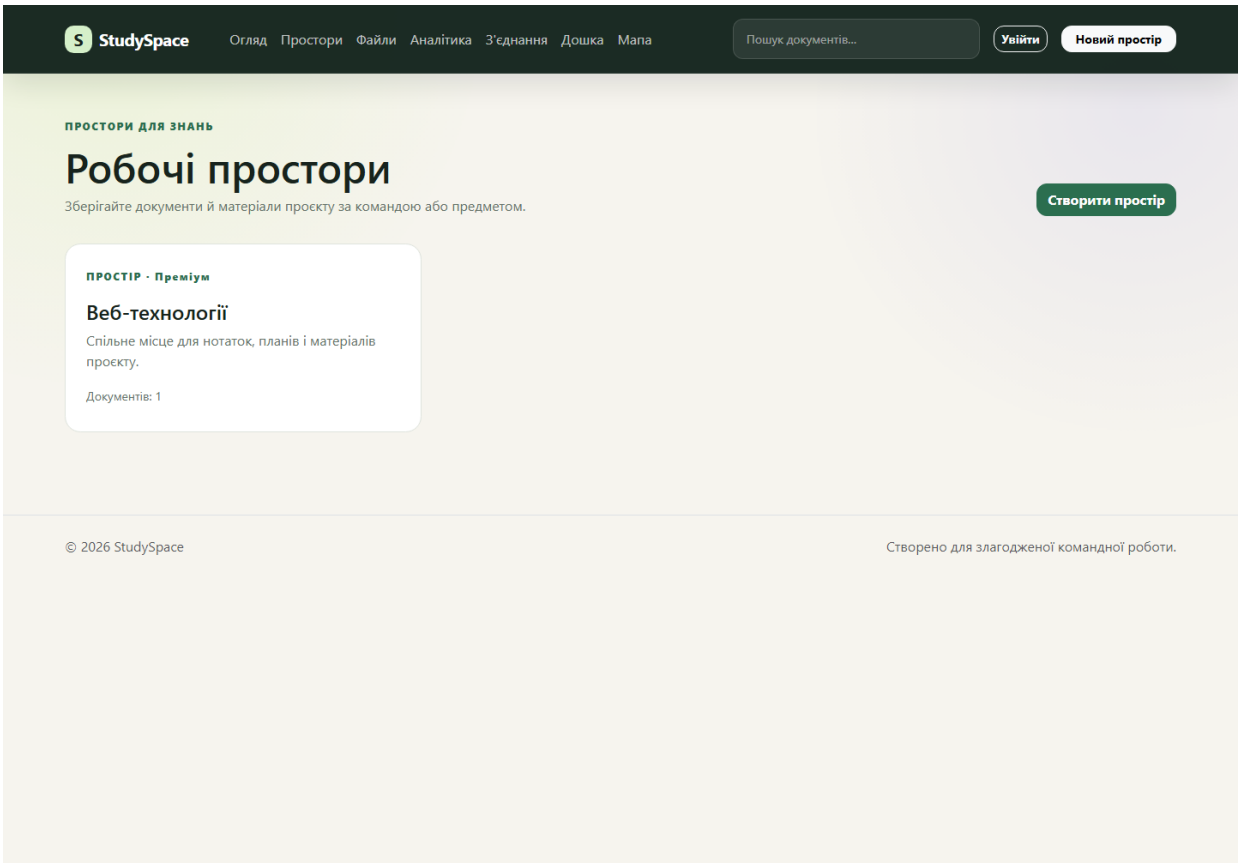


Рис. 3: Список доступних робочих просторів

4.3. Крок 3. Конкретний workspace

Власник бачить документи й дозволені дії: створення, налаштування, спільну дошку та асистента.

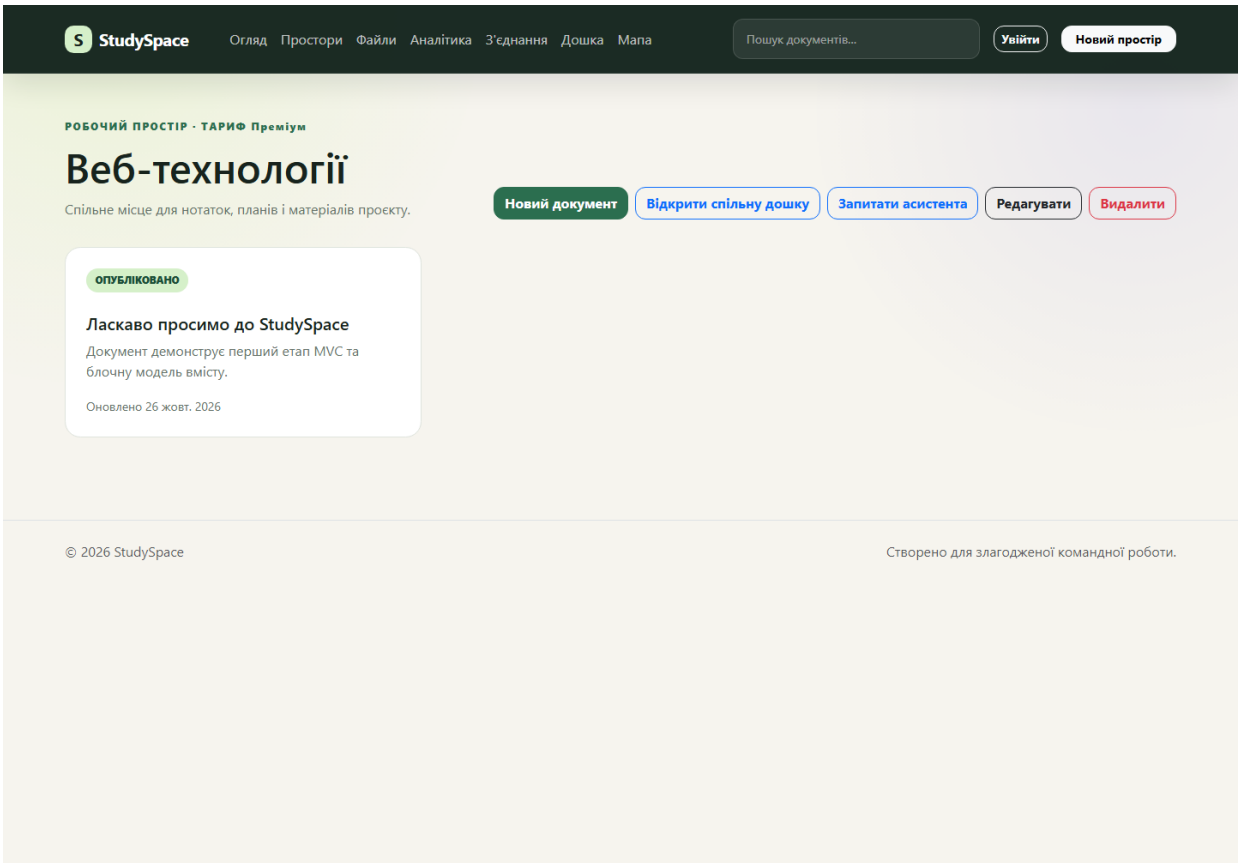


Рис. 4: Деталі робочого простору

4.4. Крок 4. Створення документа

Форма приймає назву, короткий опис, URL обкладинки й статус. Сервер перевіряє модель, доступ та унікальність, після POST виконує redirect і, за наявності налаштувань, telemetry, RabbitMQ та webhook.

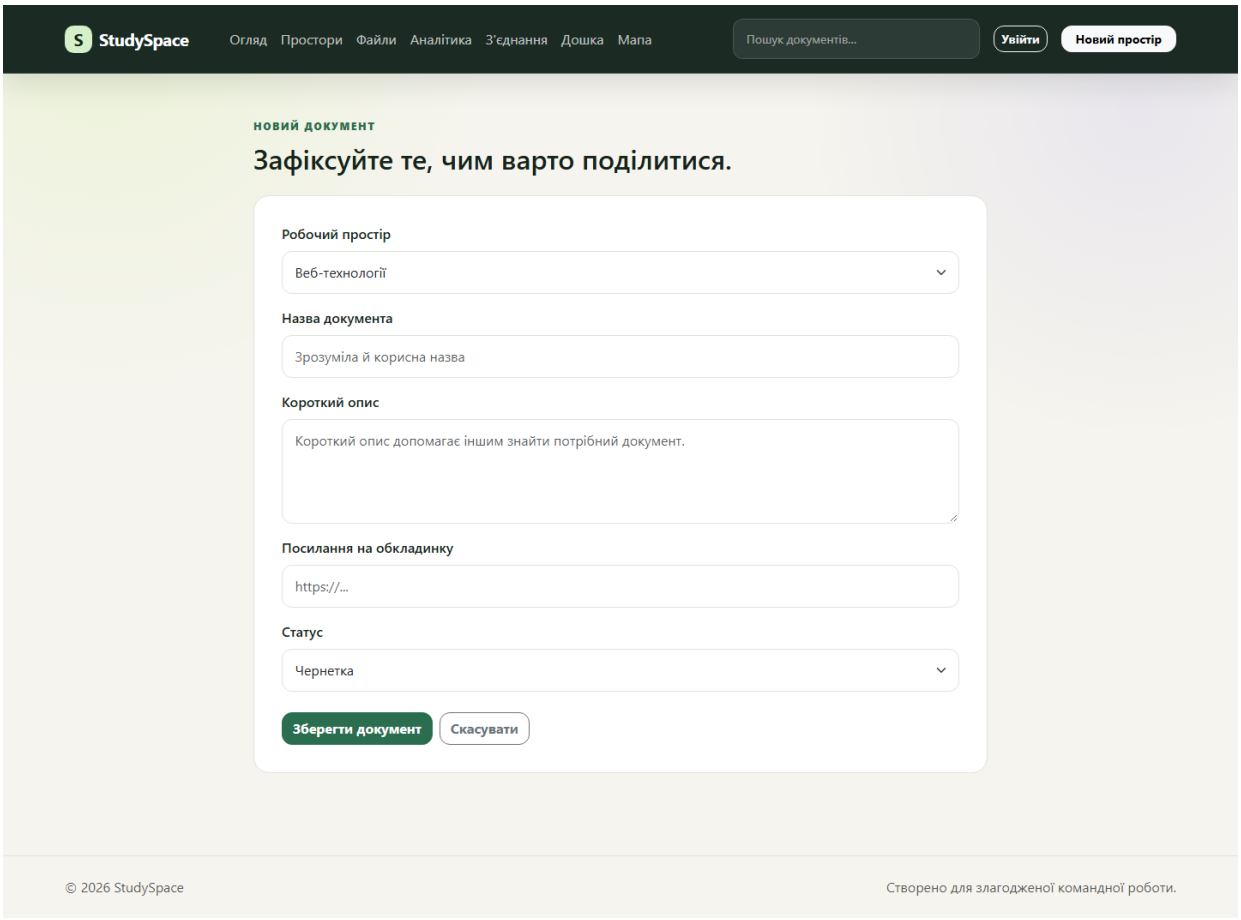


Рис. 5: MVC-форма створення документа

4.5. Крок 5. Деталі документа

Сторінка об’єднує відомості, блоки та вкладення. Звідси відкриваються редактор, редагування та завантаження файлів.

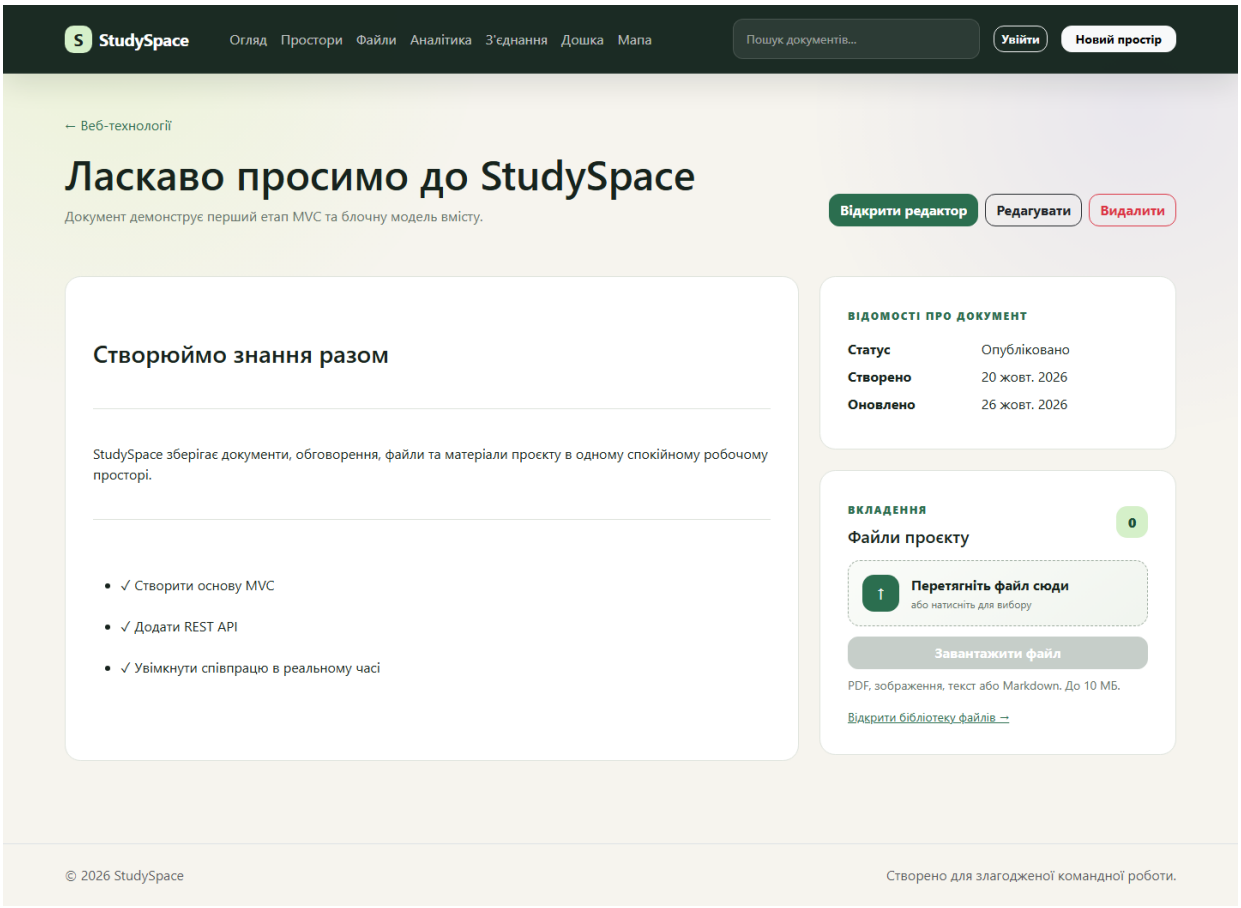


Рис. 6: Документ із контентом і формою вкладень

4.6. Крок 6. Блоковий редактор

Заголовок, абзац, список, код і цитата зберігаються через REST після короткої затримки. Перетягування змінює порядок, SignalR передає зміни та фокус.

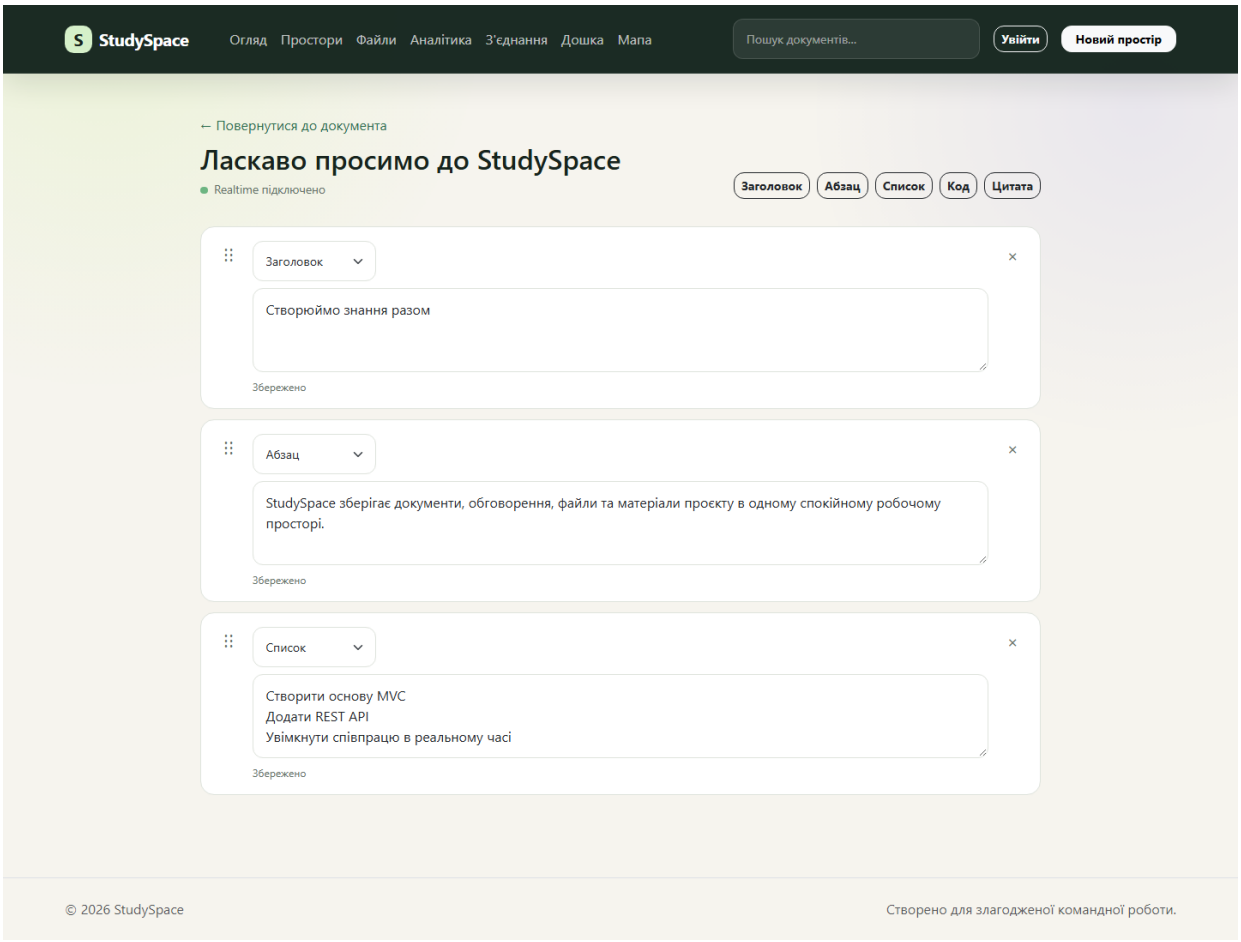


Рис. 7: Блоковий realtime-редактор

4.7. Крок 7. Файли

Зона завантаження читає DataTransfer.files, FileReader будує локальний перегляд, сервер перевіряє розширення й тарифний ліміт.

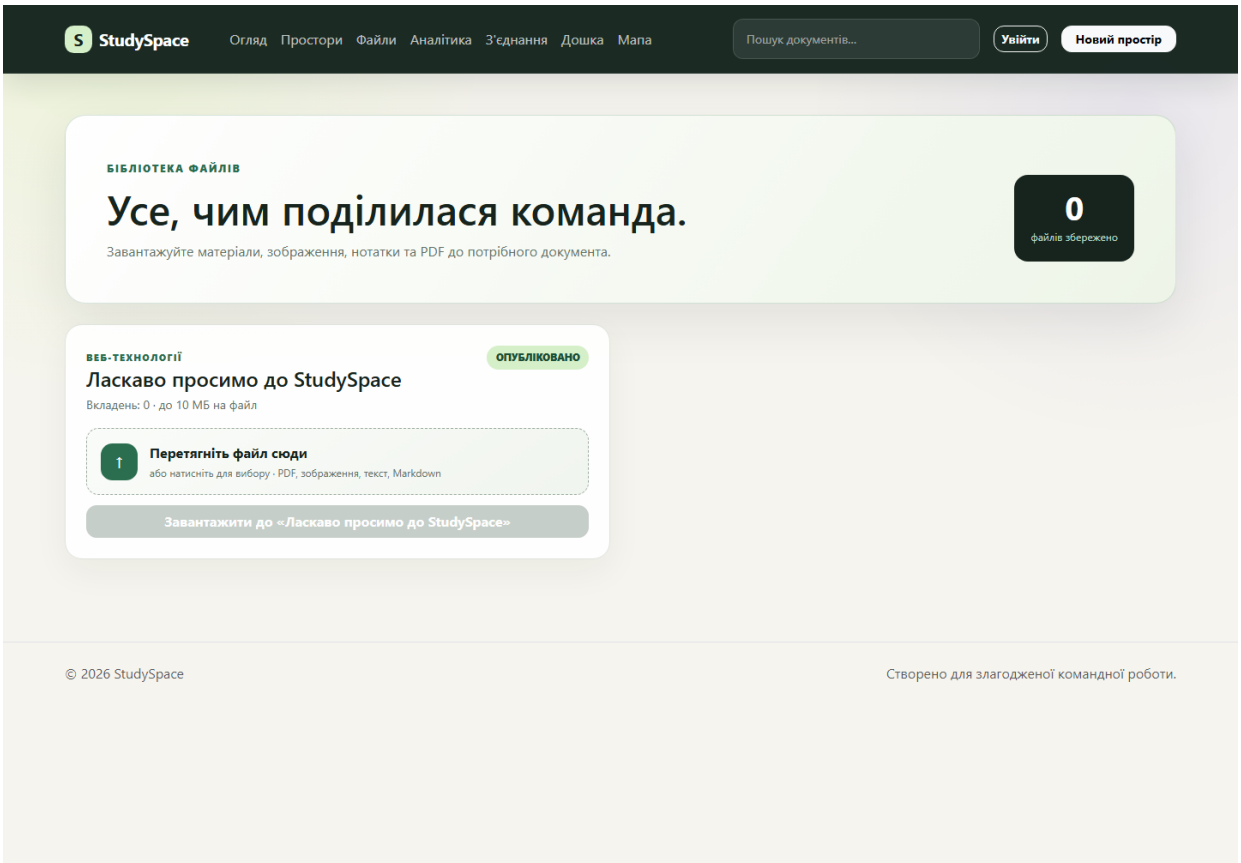


Рис. 8: Бібліотека до завантаження

4.8. Крок 8. Аналітика

Сторінка аналітики рахує дані з БД; Chart.js будує діаграму статусів.

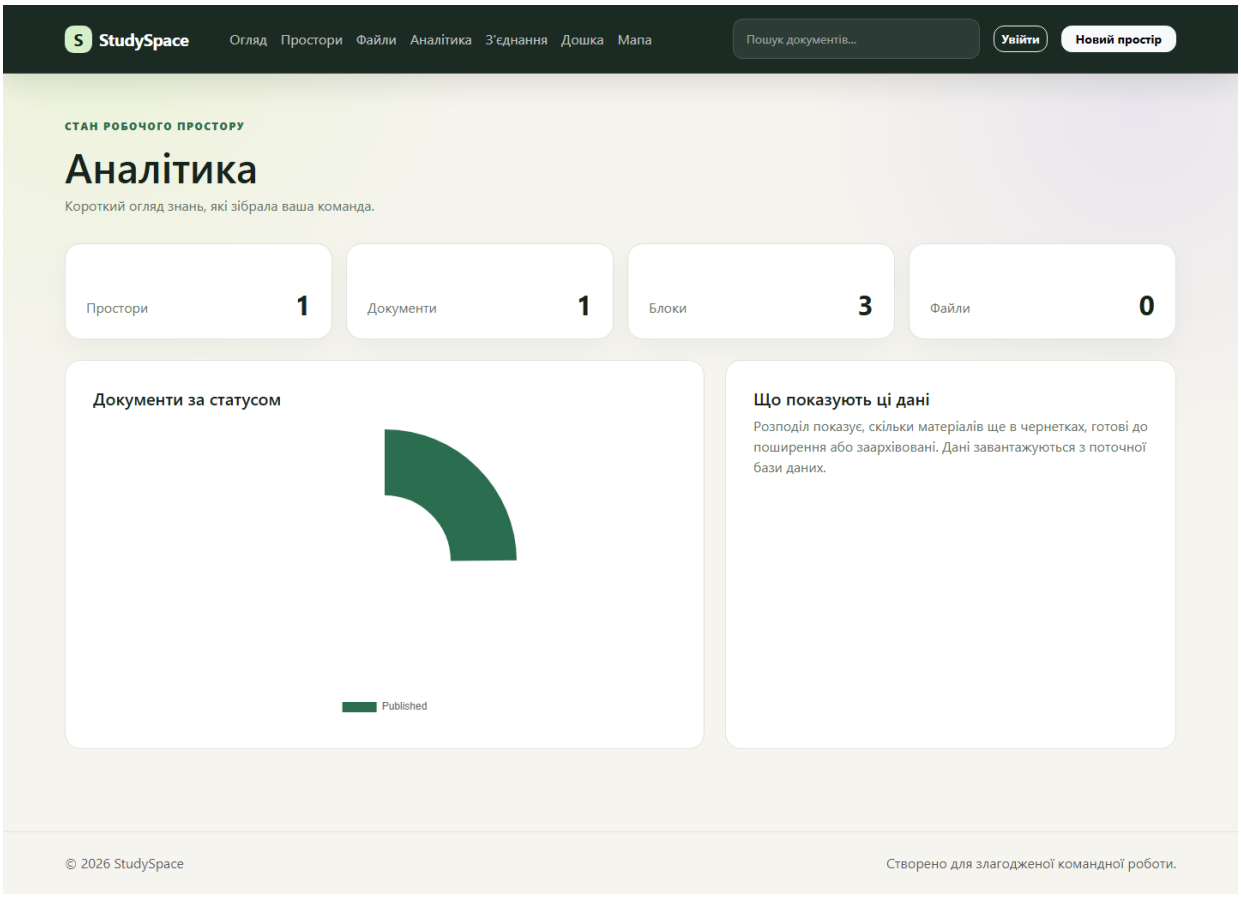


Рис. 9: Аналітика з Chart.js

4.9. Крок 9. REST API

Swagger дозволяє виконати CRUD, пагінацію, кешування та пошук. Інтеграційні тести перевіряють HTTP-коди та JSON-контракт.

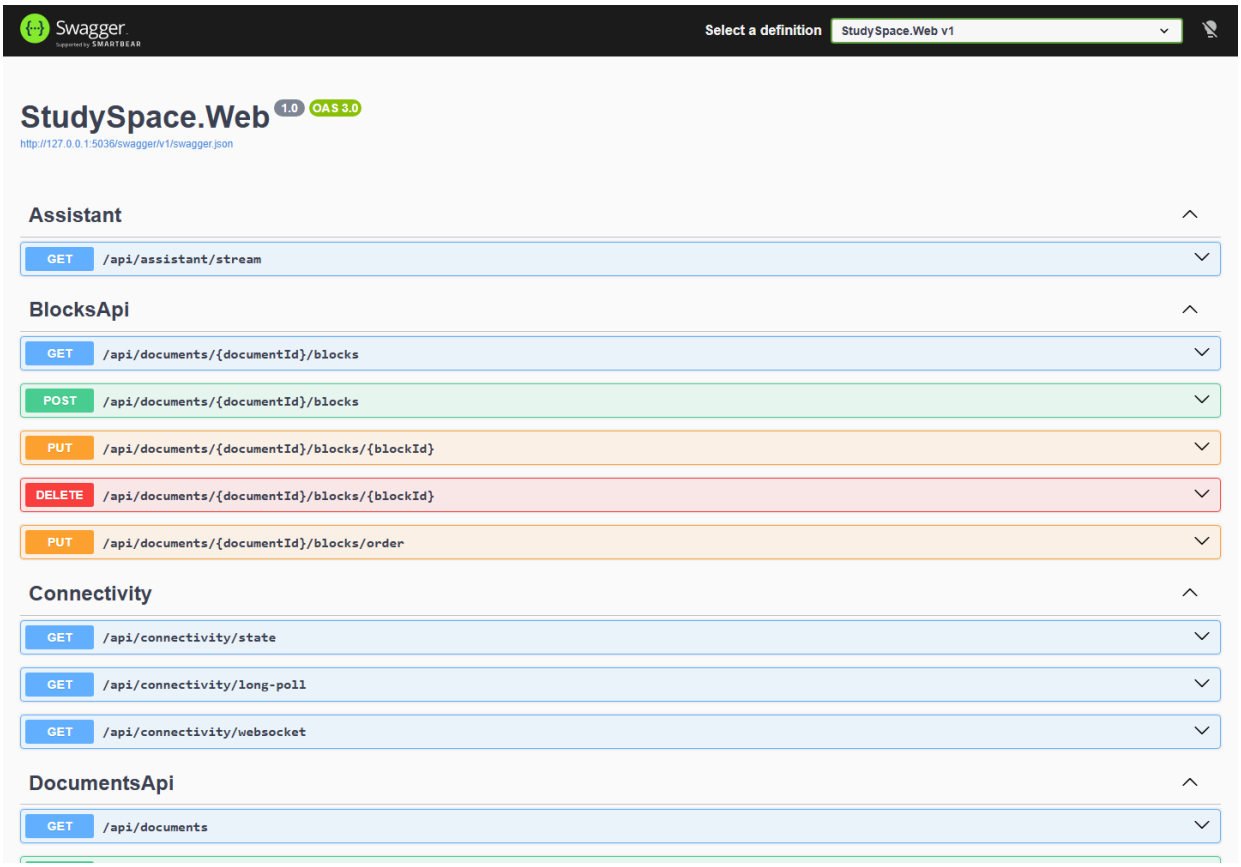


Рис. 10: Swagger/OpenAPI endpoints

4.10. Крок 10. Мережеві механізми

Один сигнал паралельно читається polling, long polling і WebSocket. Екран рахує запити, повідомлення й затримку.

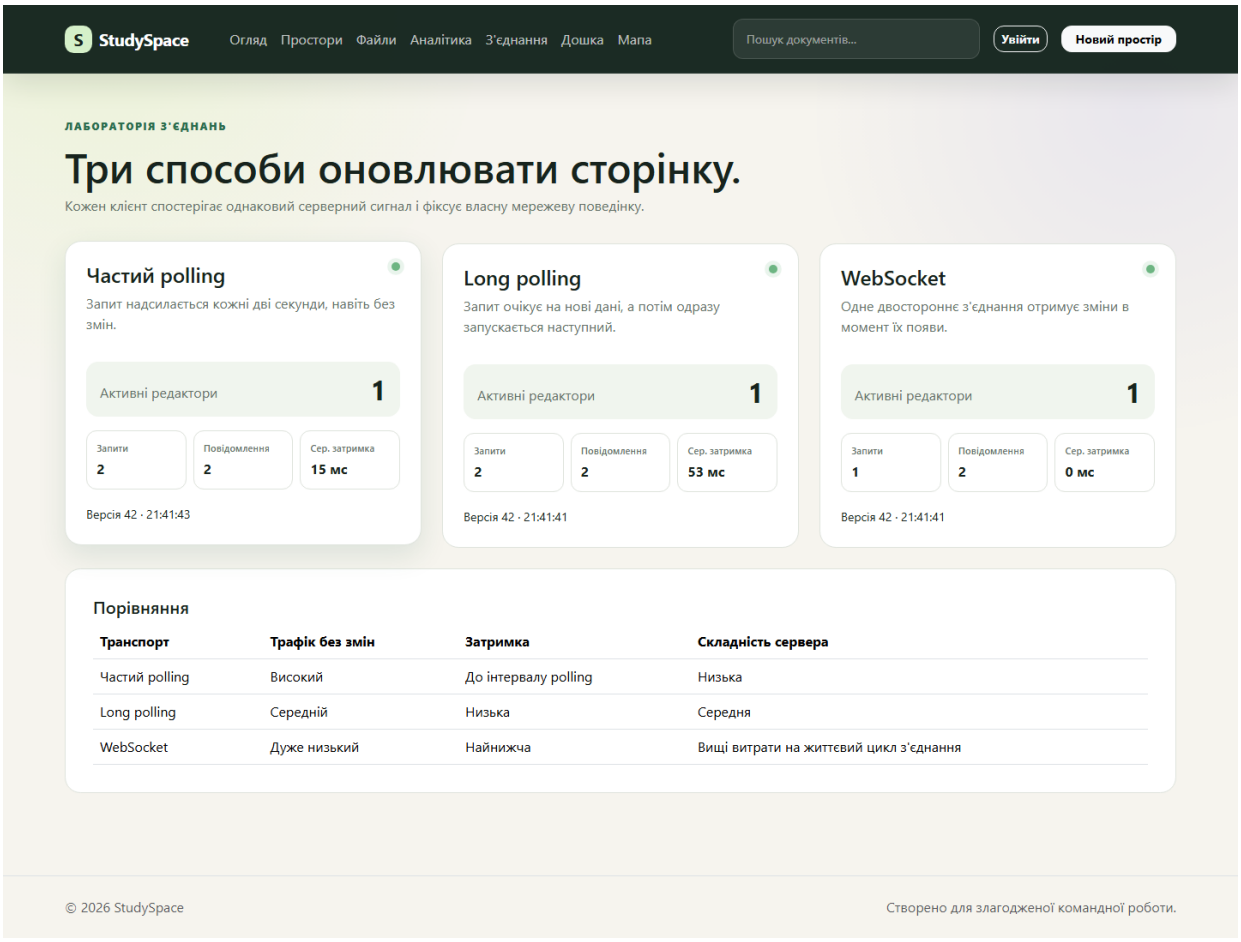


Рис. 11: Порівняння трьох transport

4.11. Крок 11. Спільна дошка

Canvas 2D реагує на Pointer Events, координати нормалізуються, штрихи та очищення синхронізуються SignalR.

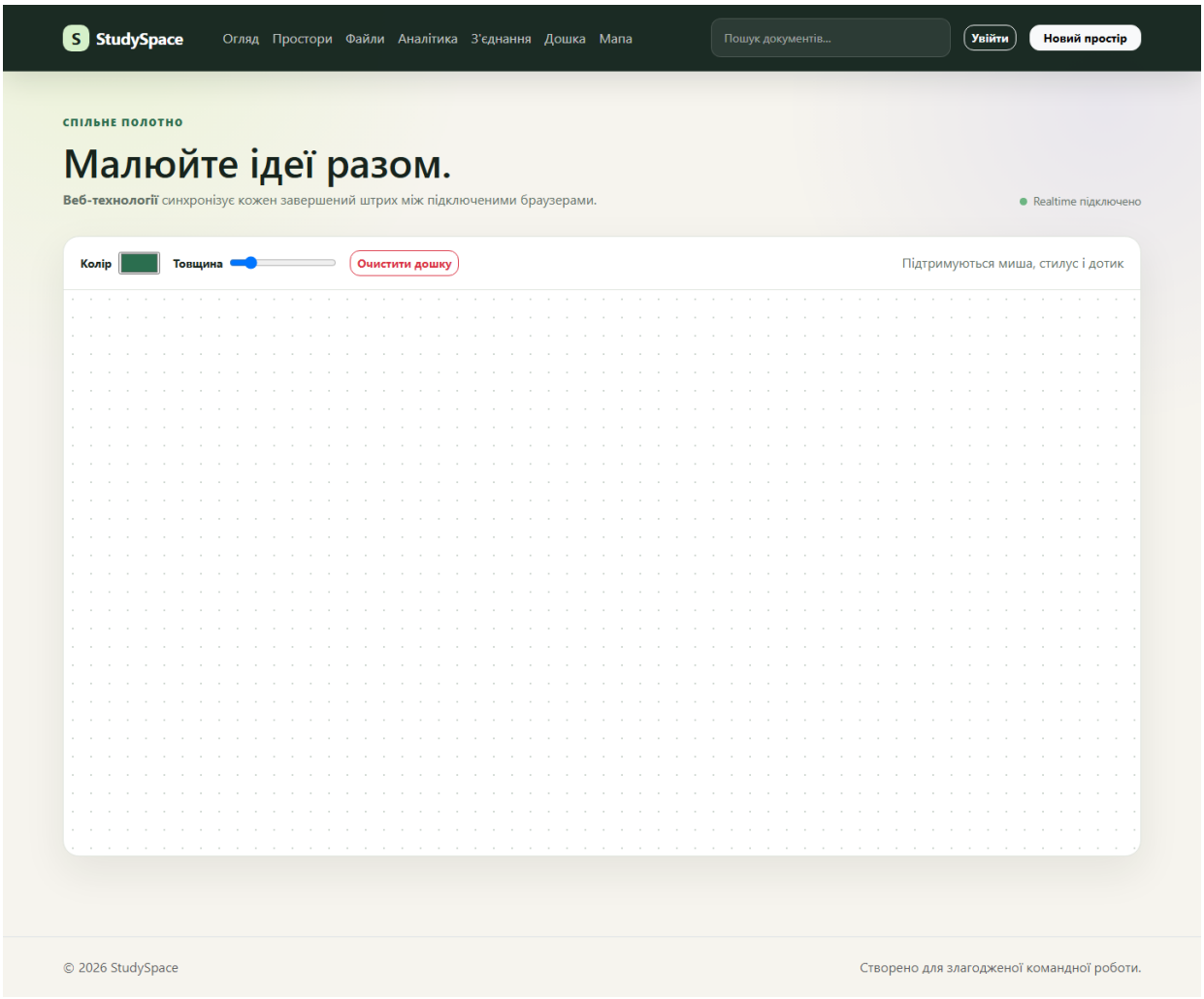


Рис. 12: Спільна дошка в режимі realtime

4.12. Крок 12. Карта

Leaflet завантажує маркери доступних просторів, OpenStreetMap надає карту, а фільтр за тарифом повторно запитує API.

4.13. Крок 13. SSE-помічник

Помічник передає етапи пошуку й відповідь частинами через SSE. Без API key працює локальний контекстний варіант.

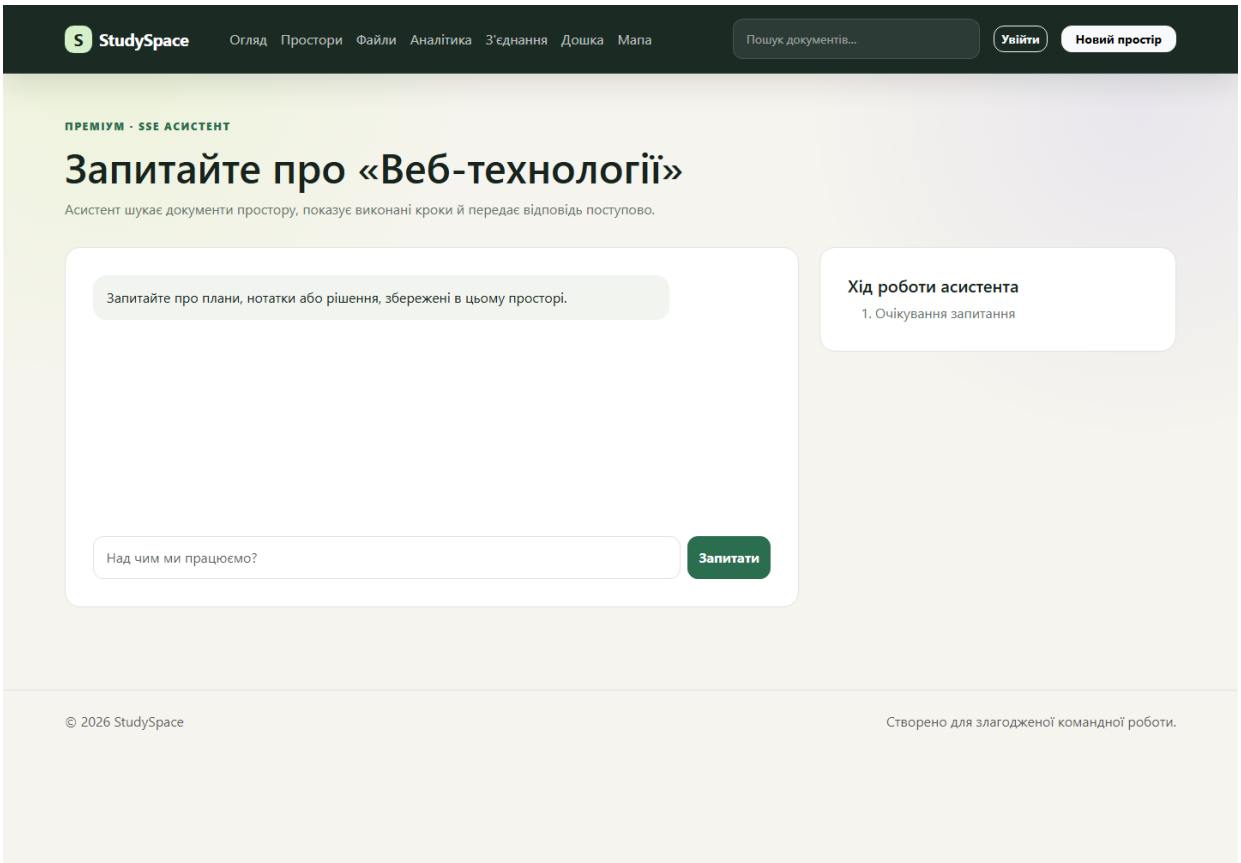


Рис. 13: Асистент робочого простору

5. Детальний опис 36 завдань

5.1. Common

5.1.1. C1: MVC і модель даних (3 б.)

Мета. Створити пов’язаний CRUD із файлом.

Реалізація і перевірка. У Core створено п’ять сутностей, у Data створено DbContext, міграції й seed. WorkspacesController і DocumentsController реалізують CRUD, AttachmentStorage файли, CoverImageUrl містить зовнішнє посилання. Перевірено всі операції та повторний запуск без дублювання seed.

5.1.2. C2: Azure deployment (1 б.)

Мета. Підготувати cloud-публікацію.

Реалізація і перевірка. Workflow виконує Release publish і conditional azure/webapps-deploy. Назва ресурсу та publish profile винесені у vars/secrets. Release build пройшов; живий URL потребує особистої Azure subscription.

5.1.3. C3: Continuous integration (2 б.)

Мета. Автоматизувати перевірки змін.

Реалізація і перевірка. `ci.yml` має jobs для restore, format, build, tests, container, frontend та E2E. TRX і Playwright report публікуються як artifacts. Ключові команди відтворено локально.

5.1.4. C4: Контейнеризація (2 б.)

Мета. Описати web, worker і broker.

Реалізація і перевірка. Multi-stage Dockerfiles створюють Web і Worker images, Compose під'єднує RabbitMQ і volumes. Файли перевірено статично та збіркою проєктів; Docker Desktop відсутній, тому runtime Compose слід перевірити після інсталяції.

5.1.5. C5: REST API (3 б.)

Мета. Надати CRUD документів.

Реалізація і перевірка. API повертає DTO. POST дає 201+Location, PUT 200, DELETE 204; є 400, 403, 404 і 409. Lifecycle пройшов integration і ручний smoke test, тимчасовий запис видалено.

5.1.6. C6: Адаптивний UI (3 б.)

Мета. Створити зрозумілий dashboard.

Реалізація і перевірка. Bootstrap, responsive grid, cards і forms формують єдину design system. Навігація об'єднує модулі. Landing і переходи перевірено Playwright.

5.1.7. C7: SEO (2 б.)

Мета. Додати засоби індексації.

Реалізація і перевірка. Є description, canonical, Open Graph, dynamic robots.txt і sitemap.xml. Routes покриті E2E; зовнішній SEO audit можливий після deployment.

5.1.8. C8: Карта (3 б.)

Мета. Відобразити геодані й фільтр.

Реалізація і перевірка. Map/API повертають доступні координати; Leaflet показує OpenStreetMap і markers, plan selector фільтрує дані. Marker перевірено E2E і скріншотом.

5.1.9. C9: Application Insights (2 б.)

Мета. Підготувати telemetry.

Реалізація і перевірка. SDK активується лише з connection string; document create/update фіксують custom events. Без секрету використовується безпечний no-op; Portal потребує власного ресурсу.

5.1.10. C10: Polling, long polling, WebSocket (4 б.)

Мета. Практично порівняти transport.

Реалізація і перевірка. ConnectivityController реалізує state, очікування нової версії та WebSocket. Клієнт рахує запити, повідомлення й latency. Локально всі три канали працювали.

5.1.11. C11: SignalR (4 б.)

Мета. Додати високорівневу realtime-взаємодію.

Реалізація і перевірка. CollaborationHub має groups за document/board id і передає blocks, focus, strokes та clear. Editor і Board показали Realtime connected.

5.1.12. C12: Блоковий редактор (3 б.)

Мета. Створити Notion-подібний контент.

Реалізація і перевірка. DocumentBlock має type/content/position. API та UI підтримують п’ять типів, debounced save і persistence після refresh.

5.1.13. C13: Realtime content (2 б.)

Мета. Передавати зміни без reload.

Реалізація і перевірка. Після API-операції hub транслює create/update/delete/order. Власна подія не дублюється, remote update підсвічується. Blocks API і browser flow протестовані.

5.1.14. C14: Presence і focus (2 б.)

Мета. Показати блок іншого редактора.

Реалізація і перевірка. focus/blur передають document id, block id та identity. Group isolation не дозволяє подіям перейти в інший документ.

5.1.15. C15: File API (3 б.)

Мета. Застосувати File, DataTransfer, FileReader.

Реалізація і перевірка. Drop читає DataTransfer.files, другий DataTransfer наповнює native input, FileReader дає preview. Сервер незалежно перевіряє size/extension. Реальний text upload пройшов.

5.1.16. C16: OAuth 2.0 (3 б.)

Мета. Додати зовнішній sign-in.

Реалізація і перевірка. Google handler реєструється лише з ClientId/Secret, callback: `/signin-google`. Для автономного показу є development cookie switcher. Реальний consent потребує Google credentials і HTTPS callback.

5.1.17. C18: Realtime shared board (4 б.)

Мета. Створити спільне малювання.

Реалізація і перевірка. BoardStateService тримає bounded history, SignalR передає draw/clear, новий клієнт отримує snapshot. Локально перевірено підключення.

5.1.18. C19: Cloud/local storage (3 б.)

Мета. Абстрагувати attachments.

Реалізація і перевірка. AttachmentStorage обирає Azure Blob за connection string або local uploads. БД зберігає URL та metadata. Local path перевірено фактичним upload; Azure потребує storage account.

5.1.19. C21: SSE-помічник (5 б.)

Мета. Потоково передавати прогрес і відповідь.

Реалізація і перевірка. Assistant повертає planning, tool, token, done і використовує tenant context. Smoke test отримав 200 та done=true; UI показав agent flow.

5.1.20. C22: MCP server (5 б.)

Мета. Надати стандартні tools.

Реалізація і перевірка. Окремий stdio server підтримує initialize, discovery, tools/list і tools/call. Smoke tools/list повернув list_documents, search_documents, get_document.

5.2. BackEnd

5.2.1. B1: Пагінація (2 б.)

Мета. Обмежити collection response.

Реалізація і перевірка. GET приймає skip/limit, повертає items, total і nextLink; limit 1–100. Smoke test: total=1, items=1; validation і nextLink покриті тестами.

5.2.2. B2: Webhook/Telegram (2 б.)

Мета. Надсилати publish notification.

Реалізація і перевірка. WebhookNotificationService підтримує generic webhook і Telegram Bot API, не ламаючи CRUD при outage. Код збирається; доставка потребує token/chat id або test webhook.

5.2.3. B4: Full-text search (2 б.)

Мета. Реалізувати релевантний пошук.

Реалізація і перевірка. DocumentSearchService будує tenant-scoped Lucene index і scored results для AJAX autocomplete. Smoke знайшов 1 результат; UI покрито E2E.

5.2.4. B5: Ролі (3 б.)

Мета. Розділити Owner, Editor і Viewer.

Реалізація і перевірка. TenantAccessService перевіряє membership і minimum role. Viewer читає, Editor змінює контент, Owner керує простором, учасниками та планом.

5.2.5. B6: Multi-tenancy і plans (3 б.)

Мета. Ізолювати команди й feature gates.

Реалізація і перевірка. UI/API/search/map/cache/SSE обмежені membership. Free: 2 MB; Premium: 10 MB, board, assistant, advanced insights. Unit/auth tests доводять правила.

5.2.6. B7: Azure Functions (3 б.)

Мета. Створити serverless API.

Реалізація і перевірка. DigestFunctions має anonymous health і function-key digest. Проєкт збирається. func CLI відсутній, тому local host/cloud runtime потребує окремої перевірки.

5.2.7. B8: RabbitMQ worker (4 б.)

Мета. Винести фонову обробку.

Реалізація і перевірка. Producer публікує durable messages, Worker має prefetch=1, ACK/reject і reconnect. Обидва проєкти збираються; broker runtime виконується окремо через Docker Compose.

5.2.8. B9: API cache (3 б.)

Мета. Кешувати повторне читання.

Реалізація і перевірка. GET by id використовує IMemoryCache на 2 хвилини. Перший response має MISS, другий HIT; PUT/DELETE інвалідують entry. Smoke і integration test пройшли.

5.3. FrontEnd

5.3.1. F1: Chart.js (3 б.)

Мета. Візуалізувати статистику.

Реалізація і перевірка. Insights агрегує актуальні DB values й передає dataset у doughnut chart. Локально видно 1 workspace, 1 document, 3 blocks і published status.

5.3.2. F2: jQuery autocomplete (2 б.)

Мета. Шукати без reload.

Реалізація і перевірка. Після 3 символів і 250 ms debounce викликається search API, попередній request скасовується, результати є links. Перевірено Playwright.

5.3.3. F3: Drag-and-drop (2 б.)

Мета. Змінювати порядок блоків.

Реалізація і перевірка. Native drag events формують масив id; server транзакційно перевіряє блоки, оновлює Position і надсилає порядок SignalR.

5.3.4. F5: React SPA (3 б.)

Мета. Створити окремий TypeScript-клієнт.

Реалізація і перевірка. React/Vite SPA має list, search, edit, loading/error/empty states. pnpm build успішно створив bundle: JS приблизно 223 kB (69.8 kB gzip), CSS 3.2 kB.

5.3.5. F6: Pointer Events і Canvas (5 б.)

Мета. Підтримати mouse, pen і touch.

Реалізація і перевірка. Canvas слухає pointerdown/move/up, зберігає normalized coordinates, colour і width; resize не зміщує strokes, completed stroke передається hub.

5.4. QA

5.4.1. QA1: API integration tests (2 б.)

Мета. Автоматизувати REST lifecycle.

Реалізація і перевірка. DocumentsApiTests перевіряє list/create/read/update/delete, paging, validation, not found, conflict і cache. Усі integration tests пройшли.

5.4.2. QA2: Browser E2E (3 б.)

Мета. Перевірити справжні user flows.

Реалізація і перевірка. Playwright покриває landing, Lucene autocomplete, block editor, map marker, robots.txt і sitemap.xml. Фінальний результат: 5/5 passed.

5.4.3. QA3: Authorization tests (3 б.)

Мета. Довести tenant isolation.

Реалізація і перевірка. Дві test identities показують: owner бачить seed workspace, non-member не може list/open. FeatureGateTests покриває Free/Premium.

6. REST API і ручна перевірка

Method	URL	Результат
GET	/api/documents	Paginated summaries
GET	/api/documents/{id}	Detail + X-Cache

Method	URL	Результат
POST	/api/documents	201 Created + Location
PUT	/api/documents/{id}	Update + invalidation
DELETE	/api/documents/{id}	204 No Content
GET	/api/search/documents	Lucene results
GET	/api/map/workspaces	Markers
GET	/api/connectivity/state	Pulse
GET	/api/connectivity/long-poll	Wait for version
GET	/api/connectivity/websocket	WebSocket upgrade
GET	/api/assistant/stream	SSE events

Ручний smoke test на живому сервері: collection total=1/items=1; два GET одного документа дали MISS, потім HIT; POST повернув 201; search повернув 1 result; DELETE повернув 204; Map повернув 1 marker; SSE повернув 200 і done=true.

Лістинг 2: Paginated response

```
{"items":[...],"skip":0,"limit":20,"total":1,"nextLink":null}
```

7. Безпека і конфігурація

Cookie authentication є базовою. Development switcher дає дві identity без секретів. Google OAuth активується лише з credentials. Identity зіставляється з WorkspaceMember; роль не приймається з browser request без перевірки.

- server-side validation доповнює client validation;
- anti-forgery захищає MVC state-changing forms;
- DTO запобігають mass assignment;
- tenant filters діють для UI, API, search, map, cache і SSE;
- upload має allowlist і plan-dependent limit;
- секрети не записані до Git;
- MCP stdout містить лише protocol messages.

Параметр	Призначення
Authentication: Google ClientId / ClientSecret	Google OAuth
Application Insights connection string	Azure telemetry
Storage: BlobConnectionString	Azure Blob
Notifications: Telegram token / ChatId	Telegram
Notifications: WebhookUrl	Generic webhook
RabbitMq: Host	Broker
Assistant: Endpoint / ApiKey / Model	External AI

Таблиця 7: Секретна зовнішня конфігурація

8. Тестування та результати перевірки

Перевірку виконано 26 жовтня 2026 року у Windows, .NET SDK 10.0.402, Node.js 24.22.1, pnpm 11.25.0. Сервер працював на `http://localhost:5036`; browser tests виконувалися у Chromium Playwright.

Рівні: **6 unit**, **14 integration**, **5 E2E**; разом **25 автоматизованих тестів**, усі успішні.

Перевірка	Результат	Примітка
dotnet format	PASS	Без змін
dotnet build	PASS	0 warnings, 0 errors
Unit tests	6/6 PASS	Домен і gates
Integration tests	14/14 PASS	API та authorization
React production build	PASS	TypeScript + Vite
Playwright E2E	5/5 PASS	Chromium
API CRUD/paging/cache/search	PASS	201, 204, MISS/HIT
Map/SSE/MCP	PASS	marker, done, 3 tools
File upload	PASS	Файл потім видалено

Лістинг 3: Основні команди перевірки

```
dotnet restore StudySpace.slnx
dotnet format StudySpace.slnx --verify-no-changes --no-restore
dotnet build StudySpace.slnx --no-restore
dotnet test tests/StudySpace.UnitTests --no-build
dotnet test tests/StudySpace.IntegrationTests --no-build
cd src/studyspace-spa && pnpm build
cd ../../tests/e2e && pnpm test
```

8.1. MCP smoke test

Лістинг 4: PowerShell tools/list

```
'{"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}' |
  dotnet run --project src/StudySpace.Mcp --no-build
```

Відповідь містила list_documents, search_documents і get_document.

9. Запуск кожного застосунку

9.1. Web

```
dotnet restore StudySpace.slnx
dotnet run --project src/StudySpace.Web --urls http://localhost:5036
```

9.2. React SPA

```
cd src/studyspace-spa
pnpm install
pnpm dev
```

API має працювати паралельно.

9.3. Container stack

```
docker compose up --build
```

Після інсталяції Docker: web використовує порт 8080, RabbitMQ використовує порт 5672, management UI використовує порт 15672.

9.4. Azure Functions

```
cd src/StudySpace.Functions
func start
```

Потрібні Functions Core Tools і StudySpaceApiUrl.

9.5. MCP

```
dotnet run --project src/StudySpace.Mcp
```

Це не сайт: stdin/stdout підключаються до MCP-клієнта.

10. Автоматизація перевірок

GitHub Actions автоматично виконує форматування, збірку, .NET-тести, production-збірку frontend, Playwright E2E та Docker image build у чистому Ubuntu environment.

11. Висновки

Реалізовано StudySpace і 36 пунктів на 104 бали: один монорепозиторій, 5 запускних застосунків і 10 технічних проєктів. Пов’язані MVC, API, data, auth, search, files і realtime-функції об’єднано, а React, Functions, Worker і MCP залишено окремими процесами.

Перевірка підтвердила збірку без warnings/errors, 25 тестів, React bundle, CRUD, pagination, cache MISS/HIT, Lucene, map, SSE і MCP tools/list. Роботу зафіксовано 12 актуальними скріншотами. Рішення має завершену структуру та може бути розгорнуте у зовнішньому середовищі після налаштування відповідних сервісів.